LABORATÓRIO: PROGRAMAÇÃO DE MEMÓRIA COMPARTILHADA COM OPENMP

Exercício 3 – Multiplicação de Matrizes

Faculdade de Computação e Informática – FCI
Aluno: Pedro de Souza Zequi
RA: 10419805

---

1. Introdução

Este relatório apresenta a resolução do Exercício 3 do laboratório "Explorando OpenMP": um programa de multiplicação de matrizes (C = A × B) que faz bom uso da cache (L1/L2), com uma versão sequencial e uma versão paralela em OpenMP. O desempenho da versão paralela foi medido com 1 a 6 threads, usando o tempo médio de 3 execuções, e comparado com o da versão sequencial.

2. Implementação

2.1 Uso da cache

As duas versões usam o mesmo algoritmo, com duas técnicas para aproveitar melhor a cache:

- **Ordem de laços i-k-j:** na ordem tradicional (i-j-k), a matriz B é percorrida por coluna. Como as matrizes ficam guardadas por linha na memória, cada acesso pula N posições e gera muitas faltas de cache (cache misses). Na ordem i-k-j, o laço mais interno percorre uma linha de B e uma linha de C, acessando posições vizinhas da memória (stride 1), e cada linha de cache carregada é aproveitada por inteiro.
- **Blocagem (tiling):** a multiplicação é feita em blocos de 64 × 64 elementos. Os três blocos em uso (de A, B e C) somam 3 × 64 × 64 × 8 bytes = 96 KB e cabem na cache L2. Assim, os dados são reaproveitados várias vezes enquanto ainda estão na cache, sem precisar ser buscados de novo na memória principal.

2.2 Paralelização

Na versão paralela, foi incluída a diretiva `#pragma omp parallel for schedule(dynamic)` no laço externo, que percorre os blocos de linhas de C. Cada thread calcula faixas de linhas diferentes de C, então nenhuma posição é escrita por duas threads ao mesmo tempo. Por isso não há condição de corrida, e não é necessário usar critical nem reduction. O `schedule(dynamic)` distribui as faixas sob demanda, o que equilibra a carga quando o número de faixas (2048/64 = 32) não é múltiplo do número de threads.

2.3 Metodologia

- Matrizes de 2048 × 2048 elementos do tipo double, preenchidas sempre com os mesmos valores (inteiros pequenos), o que torna o resultado exato e reprodutível.
- Conferência do resultado: a soma de todos os elementos de C (checksum) é comparada com o mesmo valor calculado por outra fórmula, $\Sigma_k (\Sigma_i A[i][k]) \cdot (\Sigma_j B[k][j])$. Se os valores forem iguais, o programa mostra "OK".
- Tempo medido com `omp_get_wtime()`, só na multiplicação. A versão sequencial foi executada 3 vezes. A paralela foi executada 3 vezes com cada número de threads, de 1 a 6.
- Compilação: `gcc -O2 -Wall -fopenmp`. Speedup = T(sequencial) / T(p threads).

3. Ambiente de execução

GitHub Codespaces com processador AMD EPYC 9V74, 2 núcleos físicos com SMT (4 threads de hardware) e 16 GB de RAM.

```
@PedroSZequi ➜/workspaces/CompParalelaOpenMP (main) $ cd /workspaces/CompParalelaOpenMP
mkdir -p ex3
nproc
lscpu | grep -E "Model name|Socket|Core|Thread"
free -h
4
Model name:                           AMD EPYC 9V74 80-Core Processor
Thread(s) per core:                   2
Core(s) per socket:                   2
Socket(s):                            1
               total        used        free      shared  buff/cache   available
Mem:            15Gi       2.2Gi       2.9Gi        61Mi        10Gi        13Gi
Swap:             0B          0B          0B
@PedroSZequi ➜/workspaces/CompParalelaOpenMP (main) $
```

Figura 1 – Ambiente de execução (nproc, lscpu e memória)

4. Execuções

```
@PedroSZequi ➜/workspaces/CompParalelaOpenMP/ex3 (main) $ ./mm_sequencial 2048
Versao: sequencial | N = 2048 | bloco = 64 | 3 repeticoes
  execucao 1: 3.5724 s
  execucao 2: 3.5460 s
  execucao 3: 3.5217 s
Media: 3.5467 s | checksum = 89308479485 | confere? OK
CSV gravado em resultado_sequencial.csv
@PedroSZequi ➜/workspaces/CompParalelaOpenMP/ex3 (main) $
```

Figura 2 – Execução da versão sequencial (N = 2048)

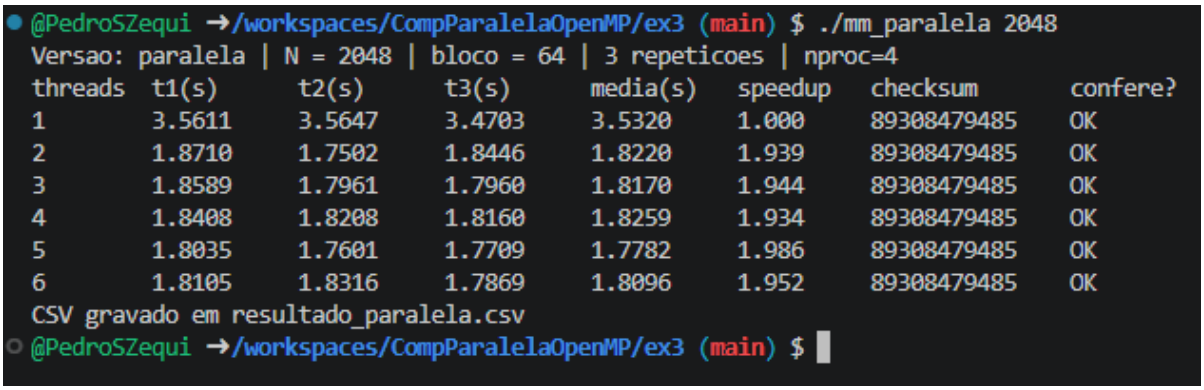


Figura 3 – Execução da versão paralela (N = 2048, 1 a 6 threads)

Todas as execuções produziram o checksum esperado, 89.308.479.485, o que confirma que as duas versões calculam a multiplicação corretamente para qualquer número de threads.

5. Resultados

A versão sequencial teve tempo médio de 3,5467 s. A tabela abaixo mostra o tempo médio da versão paralela e o speedup em relação à sequencial. A planilha Excel entregue traz os tempos individuais e o gráfico de speedup.

Tabela 1 – Versão paralela – tempo médio (3 execuções), speedup e eficiência

Threads	Tempo médio (s)	Speedup	Eficiência
1	3,5320	1,00	100,4%
2	1,8220	1,95	97,3%
3	1,8170	1,95	65,1%
4	1,8259	1,94	48,6%
5	1,7782	1,99	39,9%
6	1,8096	1,96	32,7%

6. Análise e conclusão

Com 1 thread, a versão paralela levou praticamente o mesmo tempo da sequencial (3,53 s × 3,55 s), o que mostra que o OpenMP quase não acrescenta custo extra. Com 2 threads, o tempo caiu para 1,82 s, um speedup de 1,95 e eficiência de 97%, muito perto do ideal para os 2 núcleos físicos da máquina. Isso acontece porque a multiplicação de matrizes faz muitas operações para cada dado lido e, com a blocagem, a maior parte desses dados já está na cache. O programa fica limitado pela capacidade de cálculo dos núcleos, e não pela memória, e por isso escala bem.

De 3 a 6 threads, o speedup estabilizou em torno de 2. As threads extras rodam nos núcleos lógicos do SMT, que dividem as mesmas unidades de cálculo do núcleo físico, e esse laço já ocupa essas unidades por completo. O limite do ganho, portanto, foi o número de núcleos físicos disponíveis, e não o algoritmo. Numa máquina com mais núcleos físicos, a tendência é o speedup continuar crescendo. Em

resumo, ordem de laços adequada, blocagem e um único parallel for bastaram para obter uma paralelização simples, correta e eficiente.